

## # Novocib Website - Laravel Implementation Guide

### ## Table of Contents

1. [Project Overview](#project-overview)
2. [Architecture Comparison](#architecture-comparison)
3. [Initial Setup](#initial-setup)
4. [Database Migration](#database-migration)
5. [Directory Structure](#directory-structure)
6. [Configuration](#configuration)
7. [Routing](#routing)
8. [Models & Eloquent ORM](#models--eloquent-orm)
9. [Controllers](#controllers)
10. [Views & Blade Templates](#views--blade-templates)
11. [Business Logic & Services](#business-logic--services)
12. [Form Handling & Validation](#form-handling--validation)
13. [Authentication & Authorization](#authentication--authorization)
14. [Email & Mailing](#email--mailing)
15. [Search Functionality](#search-functionality)
16. [Admin Panel](#admin-panel)
17. [Security & Encryption](#security--encryption)
18. [Logging & Error Handling](#logging--error-handling)
19. [Testing](#testing)
20. [Deployment](#deployment)

---

### ## Project Overview

The Novocib website is a PHP-based e-commerce and service provider site for biochemical products and analytical services. Currently, it uses:

- **Framework**: Vanilla PHP with custom routing
- **Database**: MySQL/MariaDB
- **Frontend**: Bootstrap, jQuery, Owl Carousel
- **Architecture**: MVC-like pattern with controllers, views, components, and repositories

### ### Current Key Features

- Product catalog with search functionality
- Contact and inquiry forms with email notifications
- Customer portal for secure payment card storage (encrypted)
- Admin interface for managing products, messages, and search index
- Customer search tracking
- 404 logging and monitoring
- Multiple service pages (enzymes, assay kits, analytical services)

---

## ## Architecture Comparison

### ### Current PHP Architecture

...

index.php (Front Controller)

- └ routes.php (Route definitions)
- └ redirects.php (Legacy redirects)
- └ Route Resolution → include views/controllers
  - └ app/views/ (Templates)
  - └ app/controllers/ (Controllers)
  - └ app/logic/ (Business logic)
  - └ app/repository/ (Data access)
  - └ app/models/ (Data objects)
  - └ app/components/ (Reusable components)
- └ app/templates/ (Layout shells)

...

### ### Proposed Laravel Architecture

...

routes/

- └ web.php (Web routes)
- └ api.php (API routes)
- └ admin.php (Admin routes)

app/

- └ Models/ (Eloquent models + relationships)
- └ Controllers/ (Request handlers)
- └ Requests/ (Form validation)
- └ Services/ (Business logic)
- └ Repositories/ (Data access - optional, or use Eloquent)
- └ Mail/ (Mailable classes)

resources/

- └ views/ (Blade templates)
- └ css/ (Compiled/SCSS)
- └ js/ (Vue/JavaScript)

database/

- └ migrations/ (Schema management)
- └ factories/ (Test data)
- └ seeders/ (Seed scripts)

...

---

## ## Initial Setup

### ### Prerequisites

- PHP 8.0+
- Composer
- MySQL 5.7+ or MariaDB 10.2+
- Node.js 14+ (for assets)

### ### Installation Steps

```bash

# 1. Create new Laravel project

composer create-project laravel/laravel novocib-laravel

# 2. Navigate to project

cd novocib-laravel

# 3. Create environment file

cp .env.example .env

# 4. Generate application key

php artisan key:generate

# 5. Create database

mysql -u root -p -e "CREATE DATABASE novocib\_laravel CHARACTER SET utf8mb4 COLLATE utf8mb4\_unicode\_ci;"

# 6. Configure .env with database credentials

# DB\_CONNECTION=mysql

# DB\_HOST=127.0.0.1

# DB\_PORT=3306

# DB\_DATABASE=novocib\_laravel

# DB\_USERNAME=root

# DB\_PASSWORD=your\_password

# 7. Install npm dependencies

npm install

# 8. Run migrations

php artisan migrate

```

---

### ## Database Migration

#### ### 1. Create Migration Files

```bash

# Articles table

php artisan make:migration create\_articles\_table

```

# Contact Messages
php artisan make:migration create_contact_messages_table

# Customers (with secure payment flow)
php artisan make:migration create_customers_table

# Encrypted Card Data
php artisan make:migration create_customer_card_data_table

# Products
php artisan make:migration create_products_table

# Search tracking
php artisan make:migration create_searches_table

# 404 logging
php artisan make:migration create_request404_logs_table

# Users (admin authentication)
php artisan make:migration create_users_table

# Pages index
php artisan make:migration create_pages_table
```

```

### ### 2. Migration Examples

**\*\*Articles Table\*\*** (`database/migrations/xxxx\_xx\_xx\_create\_articles\_table.php`):

```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('articles', function (Blueprint $table) {
            $table->id();
            $table->text('page_url')->nullable();
            $table->string('title', 255)->nullable();
            $table->longText('content')->nullable();
            $table->json('keywords')->nullable();
            $table->timestamps();
        });
    }
}
```

```

```

        public function down(): void
        {
            Schema::dropIfExists('articles');
        }
    };
    ...

```

**\*\*Products Table\*\*** (`database/migrations/xxxx\_xx\_xx\_create\_products\_table.php`):

```

```php
<?php

```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('reference', 80)->unique();
            $table->string('title', 255);
            $table->string('size', 255);
            $table->integer('price');
            $table->string('page_url', 255);
            $table->date('updated_on')->useCurrent();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
...

```

**\*\*Contact Messages Table\*\***

(`database/migrations/xxxx\_xx\_xx\_create\_contact\_messages\_table.php`):

```

```php
<?php

```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration

```

```

{
    public function up(): void
    {
        Schema::create('contact_messages', function (Blueprint $table) {
            $table->id();
            $table->string('name', 255);
            $table->string('email', 255);
            $table->string('need', 255)->nullable();
            $table->longText('message');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('contact_messages');
    }
};

```

**\*\*Customers Table\*\*** (with card reference):

```

```php
<?php

```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

```

```

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('customers', function (Blueprint $table) {
            $table->id();
            $table->string('first_name', 255);
            $table->string('last_name', 255);
            $table->string('email', 255)->unique();
            $table->string('private_id', 16)->unique();
            $table->string('password', 255)->nullable();
            $table->string('uuid', 255)->nullable();
            $table->string('company_id', 50)->nullable();
            $table->text('data')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('customers');
    }
}

```

```
}  
};  
```
```

### ### 3. Run Migrations

```
```bash  
php artisan migrate  
```
```

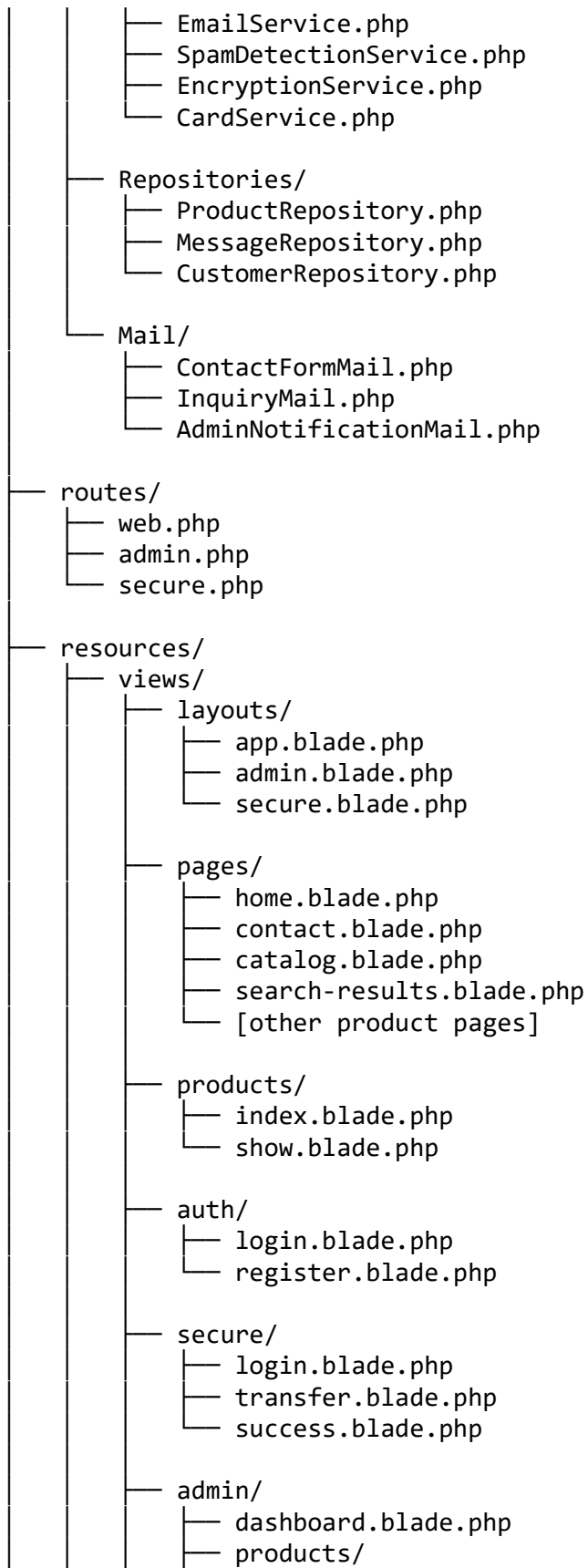
---

## ## Directory Structure

### ### Recommended Laravel Structure

```
```
```

```
novocib-laravel/  
├── app/  
│   ├── Models/  
│   │   ├── Article.php  
│   │   ├── Product.php  
│   │   ├── ContactMessage.php  
│   │   ├── Customer.php  
│   │   ├── User.php (Admin users)  
│   │   └── Search.php  
│   ├── Http/  
│   │   ├── Controllers/  
│   │   │   ├── HomeController.php  
│   │   │   ├── ProductController.php  
│   │   │   ├── SearchController.php  
│   │   │   ├── ContactController.php  
│   │   │   ├── InquiryController.php  
│   │   │   └── Secure/  
│   │   │       ├── SecureLoginController.php  
│   │   │       ├── SecureTransferController.php  
│   │   │       └── CardController.php  
│   │   ├── Requests/  
│   │   │   ├── ContactFormRequest.php  
│   │   │   ├── InquiryFormRequest.php  
│   │   │   └── SecureLoginRequest.php  
│   │   └── Middleware/  
│   │       ├── SecureFlow.php  
│   │       └── AdminAuthenticate.php  
│   └── Services/  
│       └── SearchService.php
```





```

├── messages/
├── users/
├── components/
│   ├── navbar.blade.php
│   ├── footer.blade.php
│   ├── card.blade.php
│   └── [other components]
├── emails/
│   ├── contact-form.blade.php
│   ├── inquiry.blade.php
│   └── admin-notification.blade.php
├── errors/
│   ├── 404.blade.php
│   └── 500.blade.php
├── css/ & js/
│   ├── app.css
│   ├── app.js
│   └── [existing styles]
├── database/
│   ├── migrations/
│   ├── factories/
│   ├── seeders/
│   └── sqlstate/
├── config/
│   ├── app.php
│   ├── database.php
│   ├── mail.php
│   └── encryption.php (custom)
├── .env (with secrets management)
├── .env.example
└── artisan
...

```

---

## ## Configuration

### ### 1. Environment Configuration (`.env`)

```

...`env
APP_NAME=Novocib
APP_ENV=production
APP_KEY=base64:xxxxxx

```

```
APP_DEBUG=false
APP_URL=https://novocib.com
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=novocib_laravel
DB_USERNAME=novocib_user
DB_PASSWORD=secure_password
```

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.mailer.host
MAIL_PORT=587
MAIL_USERNAME=your_email@novocib.com
MAIL_PASSWORD=mail_password
MAIL_FROM_ADDRESS=noreply@novocib.com
MAIL_FROM_NAME=Novocib
```

```
# Encryption keys for card data
CARD_ENCRYPTION_KEY=your_pbkdf2_base_key
CARD_ENCRYPTION_IV=your_initialization_vector
```
```

### ### 2. Custom Encryption Configuration (`config/encryption.php`)

```
```php
<?php
```

```
return [
    // Card data encryption settings (PBKDF2)
    'card' => [
        'algorithm' => 'sha256',
        'iterations' => 100000,
        'base_key' => env('CARD_ENCRYPTION_KEY'),
        'iv' => env('CARD_ENCRYPTION_IV'),
    ],

    // Legacy data encryption (from original PHP)
    'legacy' => [
        'method' => env('CARD_CIPHER_METHOD', 'AES-256-CBC'),
        'key' => env('CARD_CIPHER_KEY'),
        'iv' => env('CARD_CIPHER_IV'),
    ],
];
```
```

### ### 3. Logging Configuration

Ensure `config/logging.php` is set up for error tracking:

```

```php
'channels' => [
    'single' => [
        'driver' => 'single',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
    ],
    'daily' => [
        'driver' => 'daily',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
        'days' => 14,
    ],
    // Custom channels for specific logging
    'contact_forms' => [
        'driver' => 'daily',
        'path' => storage_path('logs/contact-forms.log'),
        'days' => 30,
    ],
    'security' => [
        'driver' => 'daily',
        'path' => storage_path('logs/security.log'),
        'days' => 90,
    ],
],
],
```

```

---

## ## Routing

### ### 1. Web Routes (`routes/web.php`)

```

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\ProductController;
use App\Http\Controllers\SearchController;
use App\Http\Controllers>ContactController;
use App\Http\Controllers\InquiryController;

// Public pages
Route::get('/', [HomeController::class, 'index'])->name('home');
Route::get('/home', [HomeController::class, 'index']);

// Contact and Inquiry
Route::get('/contact-us', [ContactController::class,
'show'])->name('contact.show');

```

```
Route::post('/send', [ContactController::class, 'store'])->name('contact.send');
Route::get('/inquiry', [InquiryController::class, 'show'])->name('inquiry.show');
Route::post('/send-inquiry', [InquiryController::class,
'store'])->name('inquiry.send');
```

```
// Search
```

```
Route::get('/search', [SearchController::class, 'show'])->name('search.show');
Route::post('/search', [SearchController::class,
'search'])->name('search.perform');
```

```
// Products and Services
```

```
Route::prefix('active-purified-enzymes')->group(function () {
    Route::get('/', [ProductController::class, 'enzymes'])->name('enzymes.index');
    Route::get('/human-recombinant-impdh', [ProductController::class,
'enzymeDetail'])->name('enzyme.impdh');
    // ... other enzyme routes
});
```

```
Route::prefix('freshness-assay-kits')->group(function () {
    Route::get('/', [ProductController::class,
'freshnessKits'])->name('freshness.index');
    Route::get('/fish-freshness', [ProductController::class,
'fishFreshness'])->name('freshness.fish');
    // ... other freshness routes
});
```

```
Route::prefix('analytical-services')->group(function () {
    Route::get('/', [ProductController::class,
'services'])->name('services.index');
    Route::get('/nucleotide-analysis-service', [ProductController::class,
'nucleotideAnalysis'])->name('service.nucleotide');
    // ... other service routes
});
```

```
Route::get('/catalog', [ProductController::class, 'catalog'])->name('catalog');
Route::get('/news', [ProductController::class, 'news'])->name('news');
```

```
// Secure customer flow (covered in secure.php)
```

```
// Error pages
```

```
Route::fallback(fn() => view('errors.404'))->name('404');
```

```
### 2. Secure Routes (`routes/secure.php`)
```

```
```php
```

```
<?php
```

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Secure\SecureLoginController;
```

```

use App\Http\Controllers\Secure\CardController;
use App\Http\Middleware\SecureFlow;

Route::prefix('secure')->middleware([SecureFlow::class])->group(function () {
    Route::get('/login', [SecureLoginController::class,
        'show'])->name('secure.login.show');
    Route::post('/login-c', [SecureLoginController::class,
        'store'])->name('secure.login.store');

    Route::get('/transfer', [CardController::class,
        'transferShow'])->name('secure.transfer.show')->middleware('auth:customer');
    Route::post('/store', [CardController::class,
        'storeCard'])->name('secure.card.store')->middleware('auth:customer');
    Route::get('/success', [CardController::class,
        'success'])->name('secure.success');
});

```

### ### 3. Admin Routes (`routes/admin.php`)

```

` `` php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Admin\ProductController as AdminProductController;
use App\Http\Controllers\Admin\MessageController;
use App\Http\Controllers\Admin\UserController;
use App\Http\Middleware\AdminAuthenticate;

Route::prefix('admin')->middleware(['auth',
AdminAuthenticate::class])->group(function () {
    Route::get('/', fn() => view('admin.dashboard'))->name('admin.dashboard');

    Route::resource('products', AdminProductController::class);
    Route::resource('messages', MessageController::class)->only(['index', 'show',
'destroy']);
    Route::resource('users', UserController::class);

    // Search index management
    Route::post('/search-index/rebuild', [AdminProductController::class,
'rebuildSearchIndex'])->name('admin.search-index.rebuild');
});

```

### ### 4. Route Registration (in `app/Providers/RouteServiceProvider.php`)

```

` `` php
protected function mapWebRoutes()
{
    Route::middleware('web')

```

```

        ->namespace($this->namespace)
        ->group(base_path('routes/web.php'));
    }

protected function mapSecureRoutes()
{
    Route::middleware('web')
        ->namespace($this->namespace)
        ->group(base_path('routes/secure.php'));
}

protected function mapAdminRoutes()
{
    Route::middleware(['web', 'auth'])
        ->namespace($this->namespace)
        ->group(base_path('routes/admin.php'));
}
...

```

---

## Models & Eloquent ORM

### 1. Product Model (`app/Models/Product.php`)

```

```php
<?php

```

```

namespace App\Models;

```

```

use Illuminate\Database\Eloquent\Model;

```

```

class Product extends Model
{
    protected $fillable = [
        'reference',
        'title',
        'size',
        'price',
        'page_url',
    ];

    protected $casts = [
        'updated_on' => 'date',
    ];

    public static function findByReference(string $reference): ?self
    {
        return self::where('reference', $reference)->first();
    }
}

```

```

        // Scopes
        public function scopeByCategory($query, string $category)
        {
            return $query->where('category', $category);
        }
    }
}

```

### 2. Article Model (`app/Models/Article.php`)

```

` ``php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Article extends Model
{
    protected $fillable = [
        'page_url',
        'title',
        'content',
        'keywords',
    ];

    protected $casts = [
        'keywords' => 'array',
    ];

    public static function findByUrl(string $url): ?self
    {
        return self::where('page_url', $url)->first();
    }

    // Search within content
    public function scopeSearch($query, string $searchTerm)
    {
        return $query->where('title', 'like', "%{$searchTerm}%")
            ->orWhere('content', 'like', "%{$searchTerm}%");
    }
}

```

### 3. ContactMessage Model (`app/Models/ContactMessage.php`)

```

` ``php
<?php

```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class ContactMessage extends Model
{
    protected $table = 'contact_messages';

    protected $fillable = [
        'name',
        'email',
        'need',
        'message',
    ];

    // Mark as read/unread (add boolean column if needed)
    public function markAsRead()
    {
        $this->update(['read_at' => now()]);
    }
}

```

### 4. Customer Model (`app/Models/Customer.php`)

```

```php
<?php

```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Foundation\Auth\User as Authenticatable;

class Customer extends Authenticatable
{
    protected $fillable = [
        'first_name',
        'last_name',
        'email',
        'private_id',
        'password',
        'uuid',
        'company_id',
        'data',
    ];

    protected $hidden = [
        'password',
    ];
}

```



```

        protected $casts = [
            'created_at' => 'datetime',
            'updated_at' => 'datetime',
        ];

        public function getFullNameAttribute()
        {
            return "{$this->first_name} {$this->last_name}";
        }
    }
}

```

### 5. User Model (`app/Models/User.php`) - Admin

```

` `` `php
<?php

namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable
{
    protected $fillable = [
        'username',
        'email',
        'password',
        'role',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}

```

### 6. Search Model (`app/Models/Search.php`)

```

` `` `php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

```

```

class Search extends Model
{
    protected $table = 'searches';

    protected $fillable = ['search_query'];

    public $timestamps = true;

    public static function logSearch(string $query)
    {
        return self::create(['search_query' => $query]);
    }

    public static function getPopularSearches(int $limit = 10)
    {
        return self::groupBy('search_query')
            ->selectRaw('search_query, COUNT(*) as count')
            ->orderByDesc('count')
            ->limit($limit)
            ->get();
    }
}
...

```

---

## ## Controllers

### 1. Home Controller (`app/Http/Controllers/HomeController.php`)

```

```php
<?php

```

```

namespace App\Http\Controllers;

```

```

use Illuminate\View\View;

```

```

class HomeController extends Controller
{
    public function index(): View
    {
        return view('pages.home', [
            'featuredProducts' => \App\Models\Product::limit(6)->get(),
        ]);
    }
}
...

```

### 2. Product Controller (`app/Http/Controllers/ProductController.php`)

```
```php
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Product;
use App\Models\Article;
use Illuminate\View\View;
```

```
class ProductController extends Controller
{
```

```
    public function enzymes(): View
    {
        return view('pages.products.enzymes', [
            'products' => Product::where('category', 'enzymes')->get(),
        ]);
    }
```

```
    public function enzymeDetail(string $slug): View
    {
        $article = Article::where('page_url',
            "/active-purified-enzymes/{ $slug}")->firstOrFail();
```

```
        return view('pages.products.enzyme-detail', [
            'article' => $article,
        ]);
    }
```

```
    public function freshnessKits(): View
    {
        return view('pages.products.freshness-kits', [
            'products' => Product::where('category', 'freshness')->get(),
        ]);
    }
```

```
    public function services(): View
    {
        return view('pages.services.index');
    }
```

```
    public function nucleotideAnalysis(): View
    {
        return view('pages.services.nucleotide-analysis');
    }
```

```
    public function catalog(): View
    {
        return view('pages.catalog', [
            'products' => Product::paginate(20),
        ]);
    }
```

```

    });
}

public function news(): View
{
    return view('pages.news', [
        'articles' => Article::latest()->paginate(10),
    ]);
}
}
...

```

### 3. Search Controller (`app/Http/Controllers/SearchController.php`)

```

```php
<?php

namespace App\Http\Controllers;

use App\Models\Article;
use App\Models\Search;
use App\Services\SearchService;
use App\Services\SpamDetectionService;
use Illuminate\Http\Request;
use Illuminate\View\View;

class SearchController extends Controller
{
    public function __construct(
        protected SearchService $searchService,
        protected SpamDetectionService $spamDetector,
    ) {}

    public function show(): View
    {
        return view('pages.search');
    }

    public function search(Request $request): View
    {
        $query = $request->input('q', '');

        // Spam detection - flag gibberish
        if ($this->spamDetector->isGibberish($query)) {
            Search::logSearch($query);
            return view('pages.search-results', [
                'results' => [],
                'query' => $query,
                'message' => 'No results found.',
            ]);
        }
    }
}

```

```

    }

    // Log valid search
    Search::logSearch($query);

    // Perform search
    $results = $this->searchService->search($query);

    return view('pages.search-results', [
        'results' => $results,
        'query' => $query,
    ]);
}
}
...

```

### 4. Contact Form Controller (`app/Http/Controllers/ContactController.php`)

```

```php
<?php

```

```

namespace App\Http\Controllers;

```

```

use App\Http\Requests\ContactFormRequest;
use App\Models\ContactMessage;
use App\Services\EmailService;
use App\Services\SpamDetectionService;
use Illuminate\Http\RedirectResponse;
use Illuminate\View\View;

```

```

class ContactController extends Controller
{

```

```

    public function __construct(
        protected EmailService $emailService,
        protected SpamDetectionService $spamDetector,
    ) {}

```

```

    public function show(): View
    {
        return view('pages.contact');
    }

```

```

    public function store(ContactFormRequest $request): RedirectResponse
    {
        $data = $request->validated();

        // Spam detection
        if ($this->spamDetector->detectSpam($data['message'])) {
            ContactMessage::create($data);
            return redirect()->route('contact.show')

```

```

        ->with('warning', 'Your message has been flagged for
review.');
```

```

    }

    // Save message
    $message = ContactMessage::create($data);

    // Send email
    $this->emailService->sendContactForm($message);

    return redirect()->route('message.sent')
        ->with('success', 'Your message has been sent
successfully.');
```

```

    }
}
```

```

### ### 5. Secure Flow Controllers

**\*\*SecureLoginController\*\***

(`app/Http/Controllers/Secure/SecureLoginController.php`):

```
```php
```

```
<?php
```

```
namespace App\Http\Controllers\Secure;
```

```

use App\Http\Controllers\Controller;
use App\Http\Requests\SecureLoginRequest;
use App\Models\Customer;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

```

```

class SecureLoginController extends Controller
{

```

```

    public function show(): View
    {
        return view('secure.login');
    }

```

```

    public function store(SecureLoginRequest $request): RedirectResponse
    {
        $customer = Customer::where('private_id', $request->private_id)->first();

        if (!$customer || !password_verify($request->password,
$customer->password)) {
            return back()->withErrors('Invalid credentials');
        }
    }

```

```

        Auth::guard('customer')->login($customer);

        return redirect()->route('secure.transfer.show');
    }
}
...

```

**\*\*CardController\*\*** (`app/Http/Controllers/Secure/CardController.php`):

```

```php
<?php

```

```

namespace App\Http\Controllers\Secure;

```

```

use App\Http\Controllers\Controller;
use App\Services/CardService;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

```

```

class CardController extends Controller
{

```

```

    public function __construct(protected CardService $cardService) {}

```

```

    public function transferShow(): View
    {
        return view('secure.transfer');
    }

```

```

    public function storeCard(): RedirectResponse
    {
        $customer = Auth::guard('customer')->user();

        // Store encrypted card data
        $this->cardService->storeCard($customer, request()->all());

        return redirect()->route('secure.success');
    }

```

```

    public function success(): View
    {
        return view('secure.success');
    }
}
...

```

```

---
```

**## Views & Blade Templates**

### 1. Master Layout (`resources/views/layouts/app.blade.php`)

```
````blade
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title') - Novocib</title>
    @include('components.meta-tags')
    <link rel="stylesheet" href="{{ asset('css/bootstrap.min.css') }}">
    <link rel="stylesheet" href="{{ asset('css/main.css') }}">
    <link rel="stylesheet" href="{{ asset('css/nav.css') }}">
    @stack('css')
</head>
<body>
    @include('components.navbar')

    <main class="min-vh-100">
        @yield('content')
    </main>

    @include('components.footer')

    <script src="{{ asset('js/bootstrap.bundle.min.js') }}"></script>
    <script src="{{ asset('js/app.js') }}"></script>
    @stack('js')
</body>
</html>
````
```

### 2. Home Page (`resources/views/pages/home.blade.php`)

```
````blade
@extends('layouts.app')

@section('title', 'Home')

@section('content')
    @include('components.banner')

    <section class="container my-5">
        <h1>Welcome to Novocib</h1>
        <p>Biochemical products and analytical services.</p>
    </section>

    @include('components.featured-products', ['products' => $featuredProducts])
    @include('components.services-overview')
@endsection
````
```



### 3. Product Listing (`resources/views/pages/products/enzymes.blade.php`)

```
```blade
@extends('layouts.app')

@section('title', 'Active Purified Enzymes')

@section('content')
<div class="container my-5">
    <h1>Active Purified Enzymes</h1>
    <div class="row">
        @forelse($products as $product)
            <div class="col-md-4 mb-4">
                @include('components.product-card', ['product' => $product])
            </div>
        @empty
            <p>No products found.</p>
        @endforelse
    </div>
</div>
@endsection
```
```

### 4. Contact Form (`resources/views/pages/contact.blade.php`)

```
```blade
@extends('layouts.app')

@section('title', 'Contact Us')

@section('content')
<div class="container my-5">
    <h1>Contact Us</h1>

    @if ($errors->any())
        <div class="alert alert-danger">
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form action="{{ route('contact.send') }}" method="POST">
        @csrf

        <div class="form-group mb-3">
            <label for="name">Name</label>

```

```

        <input type="text" class="form-control @error('name') is-invalid
@enderror"
            id="name" name="name" value="{{ old('name') }}" required>
        @error('name')<span class="invalid-feedback">{{ $message
    }}</span>@enderror
    </div>

    <div class="form-group mb-3">
        <label for="email">Email</label>
        <input type="email" class="form-control @error('email') is-invalid
@enderror"
            id="email" name="email" value="{{ old('email') }}" required>
        @error('email')<span class="invalid-feedback">{{ $message
    }}</span>@enderror
    </div>

    <div class="form-group mb-3">
        <label for="message">Message</label>
        <textarea class="form-control @error('message') is-invalid @enderror"
            id="message" name="message" rows="5" required>{{
old('message') }}</textarea>
        @error('message')<span class="invalid-feedback">{{ $message
    }}</span>@enderror
    </div>

    <button type="submit" class="btn btn-primary">Send</button>
</form>
</div>
@endsection
```

```

### 5. Search Results (`resources/views/pages/search-results.blade.php`)

```

```blade
@extends('layouts.app')

@section('title', 'Search Results')

@section('content')
<div class="container my-5">
    <h1>Search Results for "{{ $query }}"</h1>

    @if(isset($message))
        <p>{{ $message }}</p>
    @endif

    @foreach($results as $result)
        <div class="search-result mb-4">
            <h3>{{ $result->title }}</h3>
            <p>{{ Str::limit($result->content, 200) }}</p>
        </div>
    @endforeach
</div>
```

```

```

        <a href="{{ $result->page_url }}">Read more</a>
    </div>
    @empty
        <p>No results found for your search.</p>
    @endforelse
</div>
@endsection
```

```

### ### 6. Component Examples

**\*\*Navigation Component\*\*** (`resources/views/components/navbar.blade.php`):

```

```blade
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="{{ route('home') }}">Novocib</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item"><a class="nav-link" href="{{ route('home') }}">Home</a></li>
                <li class="nav-item"><a class="nav-link" href="{{ route('enzymes.index') }}">Enzymes</a></li>
                <li class="nav-item"><a class="nav-link" href="{{ route('search.show') }}">Search</a></li>
                <li class="nav-item"><a class="nav-link" href="{{ route('contact.show') }}">Contact</a></li>
            </ul>
        </div>
    </div>
</nav>
```

```

**\*\*Product Card Component\*\*** (`resources/views/components/product-card.blade.php`):

```

```blade
<div class="card">
    <div class="card-body">
        <h5 class="card-title">{{ $product->title }}</h5>
        <p class="card-text">Reference: {{ $product->reference }}</p>
        <p class="card-text">Size: {{ $product->size }}</p>
        <p class="card-text"><strong>${{ $product->price / 100 }}</strong></p>
        <a href="{{ $product->page_url }}" class="btn btn-primary">View Details</a>
    </div>
</div>
```

```

---

## ## Business Logic & Services

### ### 1. Search Service (`app/Services/SearchService.php`)

```
```php
<?php
```

```
namespace App\Services;
```

```
use App\Models\Article;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
class SearchService
```

```
{
```

```
    public function search(string $query): Collection
```

```
    {
```

```
        return Article::search($query)
```

```
            ->orderByRaw("MATCH(title, content) AGAINST(? IN BOOLEAN  
MODE) DESC", [$query])
```

```
            ->limit(50)
```

```
            ->get();
```

```
    }
```

```
    public function rebuildIndex(): bool
```

```
    {
```

```
        // Rebuild full-text search index
```

```
        \Illuminate\Support\Facades\DB::statement('REPAIR TABLE articles QUICK');
```

```
        \Illuminate\Support\Facades\DB::statement('OPTIMIZE TABLE articles');
```

```
        return true;
```

```
    }
```

```
}...
```

### ### 2. Spam Detection Service (`app/Services/SpamDetectionService.php`)

```
```php
<?php
```

```
namespace App\Services;
```

```
class SpamDetectionService
```

```
{
```

```
    private const GIBBERISH_THRESHOLD = 0.7;
```

```
    public function detectSpam(string $message): bool
```

```
    {
```

```
        return $this->isGibberish($message) || $this->hasSpamPatterns($message);
```

```

    }

    public function isGibberish(string $text): bool
    {
        $words = str_word_count($text, 1);

        if (count($words) < 2) {
            return true;
        }

        $vowels = preg_match_all('/[aeiouAEIOU]/', $text);
        $consonants =
preg_match_all('/[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]/', $text);

        $total = $vowels + $consonants;
        $ratio = $total > 0 ? min($vowels, $consonants) / $total : 0;

        return $ratio < self::GIBBERISH_THRESHOLD;
    }

    private function hasSpamPatterns(string $text): bool
    {
        $spamPatterns = [
            '/http[s]?:\//',
            '/viagra|cialis|casino|lottery/i',
            '/\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b/',
        ];

        foreach ($spamPatterns as $pattern) {
            if (preg_match($pattern, $text)) {
                return true;
            }
        }

        return false;
    }
}
...

```

### 3. Email Service (`app/Services/EmailService.php`)

```

```php
<?php

```

```

namespace App\Services;

use App\Mail\ContactFormMail;
use App\Mail\InquiryMail;
use App\Models\ContactMessage;
use Illuminate\Support\Facades\Mail;

```

```

class EmailService
{
    public function sendContactForm(ContactMessage $message): bool
    {
        try {
            Mail::to(config('mail.admin_email'))->send(new
ContactFormMail($message));
            return true;
        } catch (\Exception $e) {
            \Log::error('Contact form email error: ' . $e->getMessage(), ['contact'
=> 'contact-forms']);
            return false;
        }
    }

    public function sendInquiry($inquiryData): bool
    {
        try {
            Mail::to(config('mail.admin_email'))->send(new
InquiryMail($inquiryData));
            return true;
        } catch (\Exception $e) {
            \Log::error('Inquiry email error: ' . $e->getMessage(), ['contact' =>
'inquiries']);
            return false;
        }
    }
}
...

```

### 4. Encryption Service (`app/Services/EncryptionService.php`)

```

```php
<?php

namespace App\Services;

class EncryptionService
{
    private string $algorithm;
    private int $iterations;
    private string $baseKey;

    public function __construct()
    {
        $this->algorithm = config('encryption.card.algorithm');
        $this->iterations = config('encryption.card.iterations');
        $this->baseKey = config('encryption.card.base_key');
    }
}

```

```

    public function encrypt(string $data, string $salt): string
    {
        $key = hash_pbkdf2($this->algorithm, $this->baseKey, $salt,
$this->iterations, 32, true);
        $iv = openssl_random_pseudo_bytes(16);
        $encrypted = openssl_encrypt($data, 'AES-256-CBC', $key, OPENSSL_RAW_DATA,
$iv);

        return base64_encode($iv . $encrypted);
    }

    public function decrypt(string $encrypted, string $salt): ?string
    {
        try {
            $key = hash_pbkdf2($this->algorithm, $this->baseKey, $salt,
$this->iterations, 32, true);
            $data = base64_decode($encrypted, true);
            $iv = substr($data, 0, 16);
            $encrypted = substr($data, 16);

            return openssl_decrypt($encrypted, 'AES-256-CBC', $key,
OPENSSL_RAW_DATA, $iv);
        } catch (\Exception $e) {
            return null;
        }
    }
}
...

```

### 5. Card Service (`app/Services/CardService.php`)

```

```php
<?php

namespace App\Services;

use App\Models\Customer;

class CardService
{
    public function __construct(private EncryptionService $encryption) {}

    public function storeCard(Customer $customer, array $cardData): void
    {
        $salt = bin2hex(random_bytes(8));

        $encrypted = [
            'cardholder' => $this->encryption->encrypt($cardData['cardholder'],
$salt),

```

```

        'number' => $this->encryption->encrypt($cardData['number'], $salt),
        'expiry' => $this->encryption->encrypt($cardData['expiry'], $salt),
        'cvv' => $this->encryption->encrypt($cardData['cvv'], $salt),
    ];

    // Store encrypted data
    $customer->update([
        'data' => json_encode([
            'encrypted' => $encrypted,
            'salt' => $salt,
        ]),
    ]);
}

public function getCardData(Customer $customer): ?array
{
    if (!$customer->data) {
        return null;
    }

    $data = json_decode($customer->data, true);
    $salt = $data['salt'] ?? null;
    $encrypted = $data['encrypted'] ?? null;

    if (!$salt || !$encrypted) {
        return null;
    }

    return [
        'cardholder' => $this->encryption->decrypt($encrypted['cardholder'],
$salt),
        'number' => $this->encryption->decrypt($encrypted['number'], $salt),
        'expiry' => $this->encryption->decrypt($encrypted['expiry'], $salt),
        'cvv' => $this->encryption->decrypt($encrypted['cvv'], $salt),
    ];
}
}
...

---

## Form Handling & Validation

### 1. Contact Form Request (`app/Http/Requests/ContactFormRequest.php`)

```php
<?php

namespace App\Http\Requests;

```



```

use Illuminate\Foundation\Http\FormRequest;

class ContactFormRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|email|max:255',
            'need' => 'nullable|string|max:255',
            'message' => 'required|string|max:5000',
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'Name is required',
            'email.required' => 'Email is required',
            'email.email' => 'Please enter a valid email',
            'message.required' => 'Message is required',
            'message.max' => 'Message cannot exceed 5000 characters',
        ];
    }
}

```

### 2. Secure Login Request (`app/Http/Requests/SecureLoginRequest.php`)

```

` ``php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class SecureLoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {

```

```

        return [
            'private_id' => 'required|string|size:16',
            'password' => 'required|string|min:8',
        ];
    }
}
...

```

---

## ## Authentication & Authorization

### ### 1. Admin Authentication Middleware (`app/Http/Middleware/AdminAuthenticate.php`)

```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class AdminAuthenticate
{
    public function handle(Request $request, Closure $next)
    {
        $user = Auth::user();

        if (!$user || !$user->isAdmin()) {
            abort(403, 'Unauthorized access');
        }

        return $next($request);
    }
}
...

```

### ### 2. Secure Flow Middleware (`app/Http/Middleware/SecureFlow.php`)

```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class SecureFlow

```

```

{
    public function handle(Request $request, Closure $next)
    {
        // Ensure HTTPS
        if (!$request->secure() && config('app.env') === 'production') {
            return redirect()->secure($request->getRequestUri());
        }

        // Set security headers
        $response = $next($request);
        $response->header('X-Frame-Options', 'DENY');
        $response->header('X-Content-Type-Options', 'nosniff');
        $response->header('X-XSS-Protection', '1; mode=block');

        return $response;
    }
}
...

```

### ### 3. Multi-guard Authentication

Configure in `config/auth.php`:

```

```php
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
    'customer' => [
        'driver' => 'session',
        'provider' => 'customers',
    ],
],

'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => \App\Models\User::class,
    ],
    'customers' => [
        'driver' => 'eloquent',
        'model' => \App\Models\Customer::class,
    ],
],
...

---

```

## Email & Mailing

### 1. Contact Form Mailable (`app/Mail/ContactFormMail.php`)

```
```php
<?php

namespace App\Mail;

use App\Models\ContactMessage;
use Illuminate\Bus\Queueable;
use Illuminate\Mail\Mailable;
use Illuminate\Queue\SerializesModels;

class ContactFormMail extends Mailable
{
    use Queueable, SerializesModels;

    public function __construct(public ContactMessage $message) {}

    public function envelope()
    {
        return new \Illuminate\Mail\Mailables\Envelope(
            subject: 'New Contact Form Submission',
        );
    }

    public function content()
    {
        return new \Illuminate\Mail\Mailables\Content(
            view: 'emails.contact-form',
            with: [
                'message' => $this->message,
            ],
        );
    }
}
```
```

### 2. Contact Form Email Template  
(`resources/views/emails/contact-form.blade.php`)

```
```blade
<h2>New Contact Form Submission</h2>

<p><strong>Name:</strong> {{ $message->name }}</p>
<p><strong>Email:</strong> {{ $message->email }}</p>
<p><strong>Need:</strong> {{ $message->need }}</p>
<p><strong>Message:</strong></p>
<p>{{ $message->message }}</p>
```
```

<hr>

<p><small>Submitted at: {{ \$message->created\_at->format('Y-m-d H:i:s') }}</small></p>

```\n

---

## ## Search Functionality

### ### 1. Full-Text Search Migration

```php

// In ProductRepository migration or separate

```
Schema::table('articles', function (Blueprint $table) {  
    $table->fullText(['title', 'content'])->change();  
});  
```\n
```

### ### 2. Search Implementation

```php

// In SearchService

```
public function search(string $query): Collection  
{  
    return Article::whereRaw(  
        "MATCH(title, content) AGAINST(? IN BOOLEAN MODE)",  
        [$query]  
    )  
    ->orderByRaw("MATCH(title, content) AGAINST(? IN BOOLEAN MODE) DESC", [$query])  
    ->limit(50)  
    ->get();  
}  
```\n
```

---

## ## Admin Panel

### ### 1. Admin Dashboard Controller

(`app/Http/Controllers/Admin/ProductController.php`)

```php

<?php

```
namespace App\Http\Controllers\Admin;
```

```
use App\Models\Product;
```

```
use Illuminate\View\View;
```

```

use Illuminate\Http\RedirectResponse;

class ProductController extends Controller
{
    public function index(): View
    {
        return view('admin.products.index', [
            'products' => Product::paginate(20),
        ]);
    }

    public function create(): View
    {
        return view('admin.products.create');
    }

    public function store(\Illuminate\Http\Request $request): RedirectResponse
    {
        Product::create($request->validated());
        return redirect()->route('admin.products.index')->with('success', 'Product
created.');
```

created.');

```

    }

    public function edit(Product $product): View
    {
        return view('admin.products.edit', ['product' => $product]);
    }

    public function update(Product $product, \Illuminate\Http\Request $request):
RedirectResponse
    {
        $product->update($request->validated());
        return redirect()->route('admin.products.index')->with('success', 'Product
updated.');
```

updated.');

```

    }

    public function destroy(Product $product): RedirectResponse
    {
        $product->delete();
        return back()->with('success', 'Product deleted.');
```

deleted.');

```

    }

    public function rebuildSearchIndex(): RedirectResponse
    {
        app(\App\Services\SearchService::class)->rebuildIndex();
        return back()->with('success', 'Search index rebuilt.');
```

rebuilt.');

```

    }
}

```

### 2. Message Controller (`app/Http/Controllers/Admin/MessageController.php`)

```
```php
<?php

namespace App\Http\Controllers\Admin;

use App\Models\ContactMessage;

class MessageController extends Controller
{
    public function index()
    {
        return view('admin.messages.index', [
            'messages' => ContactMessage::latest()->paginate(20),
        ]);
    }

    public function show(ContactMessage $message)
    {
        return view('admin.messages.show', ['message' => $message]);
    }

    public function destroy(ContactMessage $message)
    {
        $message->delete();
        return back()->with('success', 'Message deleted.');
```

### 3. Admin Dashboard View (`resources/views/admin/dashboard.blade.php`)

```
```blade
@extends('layouts.admin')

@section('title', 'Admin Dashboard')

@section('content')
<div class="container-fluid">
    <h1>Admin Dashboard</h1>

    <div class="row">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h5>Total Products</h5>
                    <p class="h2">{{ \App\Models\Product::count() }}</p>
                </div>
            </div>
        </div>
    </div>
</div>
```

```

    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h5>New Messages</h5>
                <p class="h2">{{ \App\Models\ContactMessage::count() }}</p>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h5>Customers</h5>
                <p class="h2">{{ \App\Models\Customer::count() }}</p>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card">
            <div class="card-body">
                <h5>Recent Searches</h5>
                <p class="h2">{{ \App\Models\Search::count() }}</p>
            </div>
        </div>
    </div>
</div>
@endsection
` ``

```

---

## ## Security & Encryption

### ### 1. Encryption Configuration

Create `config/encryption.php`:

```

` `` php
<?php

return [
    'card' => [
        'algorithm' => 'sha256',
        'iterations' => 100000,
        'base_key' => env('CARD_ENCRYPTION_KEY'),
        'iv' => env('CARD_ENCRYPTION_IV'),
    ],
];
` ``

```



### ### 2. Security Headers Middleware

```
```php
// In SecureFlow middleware or new SecurityHeadersMiddleware

$response->header('Strict-Transport-Security', 'max-age=31536000;
includeSubDomains');
$response->header('Content-Security-Policy', "default-src 'self'; script-src 'self'
'unsafe-inline'");
```
```

### ### 3. Rate Limiting

```
```php
// In routes/web.php

Route::middleware('throttle:60,1')->group(function () {
    Route::post('/send', [ContactController::class, 'store']);
    Route::post('/search', [SearchController::class, 'search']);
});
```
```

---

## ## Logging & Error Handling

### ### 1. Custom Log Channels (config/logging.php)

```
```php
'channels' => [
    'contact_forms' => [
        'driver' => 'daily',
        'path' => storage_path('logs/contact-forms.log'),
        'level' => 'info',
        'days' => 30,
    ],
    'security' => [
        'driver' => 'daily',
        'path' => storage_path('logs/security.log'),
        'level' => 'warning',
        'days' => 90,
    ],
],
```
```

### ### 2. Logging in Services

```
```php
// In EmailService
```

```

\Log::channel('contact_forms')->info('Contact form sent', [
    'email' => $message->email,
    'timestamp' => now(),
]);

// In security contexts
\Log::channel('security')->warning('Failed login attempt', [
    'ip' => request()->ip(),
    'timestamp' => now(),
]);

```

### ### 3. Exception Handler (app/Exceptions/Handler.php)

```

` ``php
public function register()
{
    $this->reportable(function (Throwable $e) {
        if ($e instanceof SecurityException) {
            \Log::channel('security')->error('Security exception', [
                'message' => $e->getMessage(),
                'trace' => $e->getTraceAsString(),
            ]);
        }
    });
}

```

---

### ## Testing

#### ### 1. Feature Test Example (tests/Feature/ContactFormTest.php)

```

` ``php
<?php

namespace Tests\Feature;

use Tests\TestCase;

class ContactFormTest extends TestCase
{
    public function test_can_submit_contact_form()
    {
        $response = $this->post('/send', [
            'name' => 'John Doe',
            'email' => 'john@example.com',
            'message' => 'This is a test message',
        ]);
    }
}

```

```

        $response->assertRedirect(route('message.sent'));
        $this->assertDatabaseHas('contact_messages', [
            'email' => 'john@example.com',
        ]);
    }

    public function test_contact_form_validation()
    {
        $response = $this->post('/send', [
            'name' => '',
            'email' => 'invalid-email',
        ]);

        $response->assertSessionHasErrors(['name', 'email', 'message']);
    }
}
...

```

### ### 2. Unit Test Example (tests/Unit/SpamDetectionTest.php)

```

```php
<?php

namespace Tests\Unit;

use App\Services\SpamDetectionService;
use Tests\TestCase;

class SpamDetectionTest extends TestCase
{
    public function test_detects_gibberish()
    {
        $service = new SpamDetectionService();
        $this->assertTrue($service->isGibberish('xyzabc qwerty zzzzzz'));
    }

    public function test_accepts_valid_text()
    {
        $service = new SpamDetectionService();
        $this->assertFalse($service->isGibberish('This is a valid message'));
    }
}
...

```

---

## ## Deployment

### ### 1. Production Environment Setup

```
```bash
# 1. Clone repository
git clone <repo-url> /var/www/novocib-laravel
cd /var/www/novocib-laravel

# 2. Install dependencies
composer install --optimize-autoloader --no-dev

# 3. Create .env from .env.example
cp .env.example .env

# 4. Set up environment variables
# Edit .env with production values

# 5. Generate application key
php artisan key:generate

# 6. Run migrations
php artisan migrate --force

# 7. Seed database (if needed)
php artisan db:seed --class=ProductSeeder

# 8. Set permissions
chmod -R 775 storage bootstrap/cache

# 9. Optimize
php artisan config:cache
php artisan route:cache
php artisan view:cache

# 10. Run queue worker (if using queued jobs)
php artisan queue:work
```
```

## ### 2. Nginx Configuration

```
```nginx
server {
    listen 80;
    server_name novocib.com www.novocib.com;

    # Redirect HTTP to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name novocib.com www.novocib.com;
```

```

root /var/www/novocib-laravel/public;
index index.php;

ssl_certificate /path/to/certificate.crt;
ssl_certificate_key /path/to/private.key;

location / {
    try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
    fastcgi_pass unix:/var/run/php/php-fpm.sock;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    include fastcgi_params;
}

# Security headers
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
}
...

```

### ### 3. Supervisor Configuration for Queue Workers

```

```ini
# /etc/supervisor/conf.d/novocib-queue-worker.conf

[program:novocib-queue-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/novocib-laravel/artisan queue:work --delay=3 --tries=3
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
numprocs=4
redirect_stderr=true
stdout_logfile=/var/log/novocib-queue-worker.log
```

```

### ### 4. Cron Job for Scheduled Tasks

```

```bash
# Add to crontab
* * * * * cd /var/www/novocib-laravel && php artisan schedule:run >> /dev/null 2>&1
```

```

### ### 5. CI/CD Deployment Example (GitHub Actions)

```
```yaml
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Deploy via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.HOST }
          username: ${ secrets.USERNAME }
          key: ${ secrets.SSH_KEY }
          script: |
            cd /var/www/novocib-laravel
            git pull origin main
            composer install --optimize-autoloader --no-dev
            php artisan migrate --force
            php artisan cache:clear
            sudo systemctl restart php-fpm
...
---
```

### ## Migration Checklist

- [ ] Create all tables via migrations
- [ ] Set up models with relationships
- [ ] Create all controllers
- [ ] Create form validation requests
- [ ] Create views and blade templates
- [ ] Set up routing (web, admin, secure)
- [ ] Implement business logic services
- [ ] Configure authentication guards (admin & customer)
- [ ] Set up email configuration and mailables
- [ ] Implement encryption service
- [ ] Configure logging channels
- [ ] Create tests (feature and unit)
- [ ] Set up admin panel

- [ ] Test all forms and workflows
- [ ] Configure production environment
- [ ] Set up deployment pipeline
- [ ] Migrate data from old system
- [ ] Verify search functionality
- [ ] Test secure payment flow
- [ ] Performance optimization (caching, indexing)
- [ ] Security audit (headers, HTTPS, validation)

---

## ## Key Differences and Advantages

### ### Why Laravel Over Current Setup?

1. **Built-in ORM**: Eloquent replaces manual repository pattern
2. **Validation**: Request classes provide automatic validation
3. **Authentication**: Multi-guard support for admin and customers
4. **Security**: CSRF protection, password hashing, SQL injection prevention built-in
5. **Mailing**: Queue-able mail with templates
6. **Templating**: Blade provides powerful features vs manual PHP templates
7. **Testing**: Integrated testing framework
8. **Middleware**: Easier to manage cross-cutting concerns
9. **Database Migrations**: Version control for database schema
10. **Package Ecosystem**: Leverage thousands of verified packages
11. **Documentation**: Excellent, up-to-date documentation
12. **Community**: Larger community and more resources

---

## ## Conclusion

This Laravel implementation provides:

- A modern, maintainable codebase
- Better security out of the box
- Easier testing and debugging
- Scalability for future features
- Industry-standard framework for PHP
- Better developer experience

The migration preserves all functionality while providing a solid foundation for future enhancements.